

Checking the output for all contents in an Array:

You can know all outputs in an array by using `*` and `@`. For positional parameters `@` notation is preferred for their answers. Look in the following example.

- `[me@linuxbox ~]$ animals=("a dog" "a cat" "a fish")`
- `[me@linuxbox ~]$ for i in ${animals[*]}; do echo $i; done`
- `a`
- `dog`
- `a`
- `cat`
- `a`
- `fish`
- `[me@linuxbox ~]$ for i in ${animals[@]}; do echo $i; done`
- `a`
- `dog`
- `a`
- `cat`
- `a`
- `fish`
- `[me@linuxbox ~]$ for i in "${animals[*]}"; do echo $i; done`
- `a dog a cat a fish`
- `[me@linuxbox ~]$ for i in "${animals[@]}"; do echo $i; done`
- `a dog`
- `a cat`
- `a fish`

Here, an array of animals is created with three two-words strings. Check the effect of 4 different loops as the words are splitting. Both the notation notations `${animals[*]}` and `${animals[@]}` have similar results but when quoted they produce different results with `@` notation getting the right solution.

Determining the number of array elements:

You can use parameter expansion to check the number of elements and length of the string in an array with the following syntax:

- `[me@linuxbox ~]$ a[100]=foo`
- `[me@linuxbox ~]$ echo ${#a[@]} # number of array elements`
- `1`
- `[me@linuxbox ~]$ echo ${#a[100]} # length of element 100`
- `3`

Subscripts used status within Array:

With bash, you can find gaps in the subscripts assignment. Example:

- `${!array[*]}`